DAT 240 Final Project Report:

Campus Ride

Name: Aashma Adhikari, Abiral Shrestha, Rob Basnet, Tejaswi Amatya

Course teacher: Glenn Frode Henriksen

April 2026

University of Stavanger

Table of Contents

1. Introduction ---------- 2
2. Design ---------- 3
    2.1 Block Diagram ---------- 3
    2.2 Class Diagram ---------- 3
    2.3 Sequence Diagram ---------- 3
3. Backend ---------- 3
    3.1 Architecture Pattern ---------- 3
    3.2 Key Technical Decision ---------- 4
    3.3 What Went Wrong ---------- 5
    3.4 Testing Strategy ---------- 6
4. Frontend ---------- 5
    3.1 Architecture Pattern ---------- 6
    3.2 Key Technical Decision ---------- 7
    3.3 What Went Wrong ---------- 7
5. Summary and Contributions ---------- 8
    5.1 Summary of the Project ---------- 8
    5.2 Contributions ---------- 9
6. Conclusion ---------- 10

1. Introduction

Campus Ride is a web-based ride-sharing platform designed for students and staff at the University of Stavanger. It connects drivers who have spare seats with passengers travelling the same routes, making existing daily commutes visible, bookable, and governed.

Drivers can post ride offers, set departure and destination on an interactive map, define available seats, set a price, and specify preferences. Passengers search by location, date, and time window, filter by price and driver rating, and book with a personalised pickup and drop-off point. A dedicated administrator role oversees platform safety by approving driver applications, resolving user reports, monitoring platform-wide statistics, and adjusting operational settings such as the service fee percentage.

Each member, among four members, owned a distinct vertical track aligned to a core domain of the application. Track A covered user registration, authentication, and driver onboarding; Track B handled ride creation and the matching algorithm; Track C owned the booking lifecycle and payment simulation; and Track D handled platform governance and real-time communication. This structure proved valuable throughout. It minimised merge conflicts, gave each person end-to-end ownership of a coherent feature set, and allowed parallel progress across the team.

Task management was handled through Jira. We prompted Claude to generate tickets at the start of the project, which produced a detailed plan and broke the application into four tracks.

Tasks were divided into four sprints. Sprint 0 was dedicated to foundations. This included setting up the repo, Docker, shared schema, and CI pipeline before anyone wrote feature code. This resulted in everyone starting from the same base, reducing a lot of early friction. Sprint 1 delivered a thin vertical slice where a real user could register, post a ride, and book it end-to-end. Sprint 2 covered the full core features across all tracks, and Sprint 3 introduced advanced features, including real-time location, payment simulation, and the full notification layer.

After each sprint, the team held a scrum to review progress, discuss setbacks, and resolve all outstanding blockers before starting the next sprint. This rhythm worked well, but Sprint 2 surfaced a different problem. The team felt like they were completing tickets in isolation without confidence that everything was connecting into a coherent whole. To address this, we began using Claude to generate a structured progress file after each sprint, summarising what had been built, flagging unhandled edge cases, and identifying any features that had drifted from the original plan. This gave us a shared, honest picture of where the project actually stood rather than just what the ticket board showed. Any other communication was done through discord.

2. Design

2.1 Block Diagram

Provided is the link to the Block Diagram: Click Here

2.2 Class Diagram

Provided is the link to the Class Diagram: Click Here

2.3 Sequence Diagram

Provided is the link to the Sequence Diagram: Click Here

3. Backend

Architecture Pattern

The backend is organised around a Router → Controller → Schema pattern. Each domain including users, rides, bookings, admin, reports lives in its own folder. Each contains the same three files. The router defines the HTTP surface and attaches middleware. The controller contains the actual business logic. The schema file uses Zod to validate incoming request bodies and query parameters before anything reaches the controller.

The reason was practical. Controllers that also validate input get messy and bugs hide in that complexity. Pushing validation into Zod schemas means controllers can assume clean inputs and focus on application logic. It also made testing cheaper because schema validation can be tested in Vitest without starting the server or touching the database.

Key Technical Decisions

Authentication uses Supabase JWTs rather than session cookies. The frontend and backend run as separate containers communicating over an API, so stateless tokens are the natural fit. JWTs are self-contained. The backend can verify authenticity using the SUPABASE_JWT_SECRET without a database round-trip, reducing latency on every authenticated request. Supabase was chosen to handle token issuance and refresh rather than building a custom auth server, offloading concerns like password hashing, email verification, and token rotation. This lets the team focus on domain logic rather than the auth infrastructure. **requireAuth** validates the token and fetches the user's role from Prisma, while **requireRole** is a separate middleware applied after keeping the two concerns cleanly separated. This separation follows the single-responsibility principle; authentication and authorisation are distinct concerns. It also makes routes easy to compose. A route can be authenticated-only or authenticated and role-gated, without coupling the two checks.

Email delivery runs through a BullMQ worker rather than inside request handlers. Sending emails synchronously would delay HTTP responses and couple the success of a booking or driver approval directly to the availability of an external mail provider. If the provider is slow or down, the entire request fails. By enqueuing a job instead, the controller responds immediately, and the worker handles delivery independently in the background with automatic retry logic.

More importantly, this design decouples the domain event from its side effect. The booking controller doesn't need to know how an email gets sent, or even if it gets sent. It simply records that a job should be processed. The worker (emailWorkers.ts) subscribes to the queue and owns all delivery concerns

in isolation. This means the email implementation can be swapped out without touching any business logic. Redis acts as the message broker between the two, meaning the worker can run as a completely separate process if needed.

Real-time notifications are handled through Socket.io rather than polling or server-sent events. Booking lifecycle events are time-sensitive and bidirectional by nature. Polling would introduce unnecessary latency and server load, while SSE only supports server-to-client communication. Socket.io's room-based model was perfect for this. This would emit to scoped rooms like user:{id} and ride:{id. Events would be delivered only to the relevant driver, passenger, or ride participants without broadcasting globally. This kept the notification system both efficient and privacy-respecting. Typed payloads in socket.types.ts and an abstraction layer through notify.ts further ensure that controllers stay decoupled from the transport mechanism.

What Went Wrong

The most painful problem on the backend was schema management, and it came from a process gap rather than a technical one. Because the application was large and the full feature scope wasn't locked down at the start, the Prisma schema evolved continuously as each person added fields to support whatever they were building that week.

This caused schema drift. The app suddenly stopped being able to read user data, columns the code expected were missing from the database. Running prisma migrate deploy returned "no pending migrations," which was baffling because the database was clearly broken. What had happened was that fields were added to schema.prisma but nobody ran prisma migrate dev to actually generate the migration, so Prisma had nothing to apply and stayed silent. There was also a rogue column added directly via raw SQL at some point, which Prisma didn't know about. When we tried to force a new migration, it detected the mismatch and suggested resetting the entire database. The fix was to

manually add the missing columns via raw SQL, remove the rogue column, and rerun the migration so Prisma saw a clean state.

We learnt that one person should own a migration, it should be reviewed before merging, and prisma generate should run in CI against the actual schema, so drift gets caught early rather than when the app breaks in a shared environment.

Testing Strategy

Tests are split into two profiles. Unit tests cover business logic such as schema validation, fare calculations, and status transitions without touching any live service. Vitest blocks Prisma, Supabase, Redis, and network access in the unit profile so tests stay fast and deterministic.

Integration tests cover the infrastructure layer: middleware behaviour, HTTP routing, database interactions, queue wiring, and socket emission. These require a live database and run separately from the unit suite. GitHub Actions runs type-checking and the unit suite on every push to main, which catches regressions before they reach the shared branch.

4. Frontend

Architecture Pattern

On the frontend, we structured Campus Ride around role separation because we quickly realised that passengers, drivers, and administrators were trying to do very different things in the system. Instead of forcing all of them into one shared interface, we split the frontend into focused pages and workflows for each role. Looking back, this was one of the better decisions we made. It reduced confusion for users and also helped us collaborate as a team.

Key Technical Decisions

A major technical decision was using TanStack Query for server data. We chose it because much of the app depends on backend state that changes often, like rides, bookings, approvals, and dashboard statistics. At first,

some parts of the frontend still used useEffect and local fetching, mostly because we were learning React Query and Zustand while building the project. That made the early code a bit inconsistent. Over time, though, we saw that React Query gave us a much cleaner pattern for handling loading, errors, caching, and refetching after updates. It also made the UI more reliable, because pages could stay in sync with backend changes without each component inventing its own fetch logic. We used Zustand for a lighter shared client-side state, but one lesson from the project was that we should have agreed earlier on what belongs in React Query, what belongs in Zustand, and what should remain local UI state.

We also made a practical decision to build the interface with reusable components and a utility-first styling approach. That saved us time and helped keep the app fairly consistent even as the scope grew. Instead of treating each page as a one-off, we reused common patterns for cards, buttons, tables, and summaries, which made the frontend easier to maintain. This mattered more than we first expected, because the project grew in many directions at once. One thing that worked less smoothly was the map experience. Leaflet was fine for showing routes and ride locations, but it became awkward when we needed richer interactivity, especially things like clicking the map to select pickup and drop-off points. It was good enough for a working solution, but if we were starting again, we would probably spend more time evaluating the map layer before building so much interaction around it.

What We'd Do Differently

What we would do differently is mostly about consistency and timing. The frontend improved once we settled into clearer patterns, but we got there later than ideal. Because we were learning some of the tools while also trying to ship features, the early parts of the frontend were more mixed in style and data flow than they should have been. We also focused more on agreeing on backend contracts than on agreeing on frontend conventions, which meant some pages ended up cleaner than others. If we continued the project, we would not start by adding more features first. We would first standardise the frontend patterns more strictly, especially around server state, shared state, and interactive map behavior. Even so, the overall frontend approach worked: the role-based structure kept the app understandable, React Query made data handling much more manageable,

and reusable components gave us a foundation that held up reasonably well as the project became more complex.

5. Summary and Contributions

5.1 Summary of the project

The application is built with an Express + Prisma backend and a React + Vite frontend, communicating over a REST API and deployed together using Docker Compose for single-command startup. Authentication is handled by Supabase, with role-based access control enforced at the middleware layer. The database is seeded with realistic demo data covering all major entity states.

The team successfully implemented the core feature set across all four tracks: user registration and driver verification with document uploads, ride posting and search with map integration via Leaflet, a full booking lifecycle with confirmation and cancellation flows, and an admin dashboard covering driver approvals, report resolution, and platform statistics.

Among the advanced features, the project implements asynchronous email notifications through a BullMQ worker backed by Redis, real-time push notifications and in-ride events through Socket.io, live chat between passenger and driver and real-time driver location tracking during active rides. A CI pipeline via GitHub Actions runs type-checking and tests on every push to main, and the codebase is covered by both unit tests on schema validation and integration tests on critical request flows.

Contribution

Aashma Adhikari focused on Track C (Bookings and Payment), implementing seat booking, payment, booking cancellation, driver accept/decline flows, and booking status and earnings history.

Abiral Shrestha focused on Track A (User & Auth), covering user registration with university email, login, profile management, document uploads, driver registration and approval flow, separate admin

login, and ride ratings and reviews. He also coordinated task distribution and managed the overall workflow across the team.

Rob Basnet focused on Track B (Rides), implementing ride posting with route, time, seat, and price configuration, ride stage transitions (started, completed), location-based ride search with map integration, and live driver location via Socket.io.

Tejaswi Amatya focused on Track D (Admin & Notifications), implementing the admin dashboard with platform statistics, driver registration approvals, report and user management, platform fee configuration, asynchronous email notifications via BullMQ, Socket.io push notifications for booking events, and in-ride chat

5.2 Conclusion

Campus Ride set out to be a practical, well-structured ride-sharing platform, and by the end of the project, it delivered on that goal across all four tracks. The team navigated a non-trivial stack, real integration challenges, and the natural friction of collaborative development under deadline pressure. Not everything went perfectly. Some features were deferred, some decisions had to be revisited, and communication took time to mature but each setback led to a concrete adjustment that improved how the team worked. The experience reinforced that good software is as much about process and communication as it is about code, and that the lessons learned here around schema ownership, tooling alignment, and structured collaboration are ones that will carry forward well beyond this course.